WEEK 07

Interrupts and Exceptions

The hardware mechanism for breaking into running code. Four registers, one chain, and the sixteen cycles that make an embedded system responsive

Textbook: Dean, *Embedded Systems Fundamentals*, 2nd ed., Chapter 4, printed pages 107–122.
Lab manual: Lab 03 Section 3.7 (GPIO interrupts, EXTI, CubeMX), Lab 04 Section 4.4 (timer interrupts).

1 What we are actually after this week

Back in Week 04 we found a program that could not watch a switch while it was waiting, and in Week 05 we fixed it with an interrupt and hand-waved the configuration. This week we stop hand-waving.

The question is: **what actually happens, in hardware, when a switch closes and a function you never called starts running?**

The plan:

1. Two CPU modes, Thread and Handler, and why they exist.
2. Two stack pointers, MSP and PSP, and which one gets used when.
3. The **entry** sequence. Eight registers pushed, sixteen cycles, and what each step is for.
4. The **exit** sequence, and the surprise that ARMv6-M has no "return from interrupt" instruction.
5. The **vector table**, which is how the CPU knows which function to jump to.
6. The hardware chain: **GPIO** → **SYSCFG** → **EXTI** → **NVIC** → **CPU**. Four stages, four blocks of configuration code.
7. **Priority**, and how simultaneous requests get ordered.
8. **PRIMASK**, and the correct way to mask interrupts without breaking someone else's assumptions.

Item 6 is the exam material and the lab material, and it is the one place where the F091 and F303 genuinely differ in a way that will break copied code. Items 3 and 4 look like trivia but they explain why an ISR needs no special calling convention, which matters. Item 8 is short and it is where people write a subtle bug.

2 Deep dive 1: Two modes, and two stacks

Start with a definitional point that trips people up.

Exceptions and interrupts

> In the Arm programmer's model, an **interrupt is a type of exception**. Exceptions are triggered by hardware signals or anomalous program conditions. When one occurs, the processor: pauses the program, saves context (registers and where to resume), works out which **handler** to run, runs it, then restores context and carries on.
>
> So "exception" is the general category, covering reset, faults, and system calls. "Interrupt"

> is specifically the peripheral-driven kind. An **IRQ** (interrupt request) is the hardware signal itself.

Most interrupts are **asynchronous**, meaning they can occur almost anywhere in your program, between any two instructions. That is the whole point and also the whole danger, and Week 08 is about the danger.

2.1 Thread mode and Handler mode

The CPU has two operating modes.

- **Thread mode** is normal. Your `main` and everything it calls runs here.
- **Handler mode** is entered automatically when servicing an exception, and left when the handler finishes.

	Thread mode	**Handler mode**
When	Normal execution	Servicing an exception
Stack pointer	MSP *or* PSP, selectable	Always MSP
Entered by	Reset, or exception return	Exception processing starting

There is also a third state worth naming: **debug state**, where the CPU executes nothing and is driven by debugger hardware instead. Because that hardware has full access to registers and memory, your program does not need to be modified to be debugged, which is why single-stepping an STM32 does not require a special build.

2.2 MSP and PSP

Recall from Week 06 that the stack pointer grows downward and holds local variables and return addresses. What Week 06 did not say is that there are two of them.

- **MSP**, the main stack pointer. Used by handlers, always. The default after reset.
- **PSP**, the process stack pointer. Optionally used by Thread-mode code.

Which one `SP` refers to in Thread mode is decided by the `SPSEL` flag in the `CONTROL` register: 0 means MSP, 1 means PSP. After reset it is 0.

Why bother having two? Because of kernels. In a system with an RTOS, each task gets its own stack via PSP, while the OS and every exception handler run on MSP. That separation means a task cannot overflow its stack into the kernel's, and the kernel always has a known-good stack to work from even if a task has corrupted its own. In a bare-metal system with no kernel, only MSP is used and PSP sits idle.

In the lab

> Your labs are bare-metal, so you will use MSP throughout and can safely ignore PSP. It matters if you go on to the Embedded Systems elective and meet an RTOS, and it matters for one exam-shaped question: *which stack pointer does an ISR use?* Answer: always MSP, because an ISR runs in Handler mode.

3 Deep dive 2: Entering a handler, step by step

Here is the sequence the hardware performs when an enabled exception is requested. All of it is hardware, none of it is code you write. On a Cortex-M0 it takes **sixteen cycles**, assuming memory is fast enough not to insert wait states.

1. **Finish the current instruction**, with an exception to the exception. Five instructions are slow enough that waiting for them would hurt latency: `LDM`, `STM`, `PUSH`, `POP`, and `MULS`. Those are *abandoned* and restarted after the handler completes.
2. **Push eight 32-bit registers** onto the current stack: `xPSR`, `PC` (the return address), `LR`, `R12`, `R3`, `R2`, `R1`, `R0`. This is called the **stack frame**.
3. **Switch to Handler mode** and start using MSP.
4. **Load the PC** with the handler's address, taken from the vector table according to which exception occurred.
5. **Load LR** with an `EXC_RETURN` code, which encodes which mode and stack to return to.
6. **Load IPSR** with the exception number. For an IRQ, that number is $16 +$ IRQ number.

   Then the handler's code starts executing.

Why exactly those eight registers

> Recall the register-use convention from Week 06: `R0`–`R3` and `R12` are *caller-saved* (scratch) registers, and a called function may destroy them freely. The hardware pushes precisely the caller-saved set, plus the return address, plus `LR`, plus the flags.
>
> That is what makes an ISR an ordinary C function. The hardware has already saved everything a C function is allowed to clobber, so the compiler's normal prologue handles the rest. **You do not need any special keyword, attribute, or calling convention to write an ISR in C on Cortex-M.** On many older architectures you did.

The three `EXC_RETURN` codes, which you will see in a debugger's `LR` and might otherwise find alarming:

`EXC_RETURN`	Return stack	Meaning
`0xFFFF_FFF1`	MSP	Return to Handler mode with MSP
`0xFFFF_FFF9`	MSP	Return to Thread mode with MSP
`0xFFFF_FFFD`	PSP	Return to Thread mode with PSP

Book board vs your board

> **Interrupt latency, and why your board is faster.**
>
> Book (Cortex-M0, 48 MHz): 16 cycles from request to handler start. That is $16/48\,\text{MHz} = 333.3$ ns.
>
> Yours (Cortex-M4, up to 72 MHz): 12 cycles. At 72 MHz that is $12/72\,\text{MHz} = 167$ ns; at the 48 MHz your project plan uses, 250 ns.
>
> The Cortex-M4 also has **tail-chaining**: if a second interrupt is pending when a handler finishes, the CPU skips the pop-then-push entirely and goes straight into the next handler, at a cost of about six cycles instead of a full exit plus entry. The M0 has this too. Worth knowing because it means back-to-back interrupts are much cheaper than the naive $2\times$ estimate.

> The M4 with an FPU also stacks the floating-point registers when floating-point state is active, making the frame larger and entry slower. Relevant to your Lab 07 DSP work: if an ISR touches `float`, you pay for it on every entry.

4 Deep dive 3: Exiting a handler, and the missing instruction

Now the way out. Step 1 is software, the rest is hardware.

1. Execute an instruction that triggers exception return. **ARMv6-M has no "return from interrupt" instruction.** Instead you load the PC with the `EXC_RETURN` code, and the hardware notices. In practice that means `BX LR`, since step 5 of entry put the code in `LR`, or popping the code from the stack into the PC.
2. Based on the code, select MSP or PSP, and Handler or Thread mode.
3. Restore the eight-register frame from that stack.
4. Resume execution at the restored PC.

The trick worth appreciating

> Because the return address arrives in `LR` as a magic value with all its top bits set, and because such an address could never be a legal instruction address, the hardware can distinguish "return from a function" from "return from an exception" using the same `BX LR` instruction. No new instruction was needed. This is why a C compiler can emit an ordinary function epilogue for an ISR and have it work.

5 Deep dive 4: The vector table

So in entry step 4 the CPU loads the PC from "the vector table". What is that?

It is an array of function pointers at the very bottom of memory. Entry n holds the address of the handler for exception n. The CPU indexes into it by exception number and jumps.

Address	Vector #	Name	Description
`0x0000_0000`	—	`__initial_sp`	Initial stack pointer value
`0x0000_0004`	1	Reset	CPU reset
`0x0000_0008`	2	NMI	Non-maskable interrupt
`0x0000_000C`	3	HardFault	Illegal operation
`0x0000_002C`	11	SVCall	`SVC` instruction, supervisor call
`0x0000_0038`	14	PendSV	System-level service request
`0x0000_003C`	15	SysTick	System timer tick
`0x0000_0040`	16	IRQ0	First peripheral interrupt

Note the very first word is not a handler at all, it is the *initial stack pointer*. The CPU loads SP from address 0 and PC from address 4 before executing a single instruction, which is how a Cortex-M boots with a working stack from the first cycle.

Note also the arithmetic that reconciles two numbering schemes. Vector 16 is IRQ 0. So vector # $= 16 +$ IRQ #, which is exactly the value entry step 6 loads into `IPSR`. Two numbering systems for the same events, offset by 16, is a reliable source of confusion. The CMSIS names use IRQ numbers.

The table itself is defined in assembly in the device's startup file, using `DCD` to emit a constant word holding each handler's address:

Listing 1: From `startup_stm32f091xc.s`. Yours is `startup_stm32f303xc.s`.

```
__Vectors   DCD   __initial_sp           ; Top of Stack
            DCD   Reset_Handler
            DCD   NMI_Handler
            DCD   HardFault_Handler
            DCD   0                     ; Reserved
            ...
            DCD   SysTick_Handler
          ; External Interrupts
            DCD   WWDG_IRQHandler       ; Window Watchdog
            DCD   PVD_VDDIO2_IRQHandler
            DCD   RTC_IRQHandler
            DCD   FLASH_IRQHandler
            DCD   RCC_CRS_IRQHandler
            DCD   EXTI0_1_IRQHandler    ; EXTI lines 0 and 1
            DCD   EXTI2_3_IRQHandler    ; EXTI lines 2 and 3
            DCD   EXTI4_15_IRQHandler   ; EXTI lines 4 to 15
            ...
```

Beware the trap

The handler names are **not** arbitrary. The startup file references specific symbols, and if you define a function with exactly that name, the linker wires it into the table. If you misspell it, your function is simply never called, and there is no error, because the startup file provides a weak default handler that loops forever.

So the symptom of a typo in a handler name is: the interrupt fires, the CPU jumps to a default infinite loop, and your program appears to hang. If your board freezes the instant an interrupt should have fired, check the spelling of the handler against the startup file before you check anything else.

Book board vs your board

This is a real difference and it will break copied code.

The **F091 groups sixteen EXTI lines into three handlers**: `EXTI0_1_IRQHandler`, `EXTI2_3_IRQHandler`, and `EXTI4_15_IRQHandler`. So one function services twelve different pins and must work out which.

Your **F303 has finer granularity**: `EXTI0_IRQHandler`, `EXTI1_IRQHandler`, `EXTI2_TSC_IRQHandler`, `EXTI3_IRQHandler`, `EXTI4_IRQHandler`, then `EXTI9_5_IRQHandler` and `EXTI15_10_IRQHandler`. Lines 0 through 4 get their own handler each.

So the book's `EXTI4_15_IRQHandler` **does not exist on your part**, and its user button on PA0 maps to `EXTI0_IRQHandler` on your board. Copy the book's handler name verbatim and you get the silent-hang failure described above.

Your lab manual's example is correct for your part:

```
void EXTI0_IRQHandler(void) {
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_0);    /* HAL clears the flag  */
}
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    if (GPIO_Pin == GPIO_PIN_0) { /* your code */ }
}
```

Also worth noting: your F303's NVIC supports far more interrupt sources than the F091's

32, and 4 priority bits rather than 2. More on that in Section 7.

6 Deep dive 5: The chain, GPIO to CPU

Now the main event. A pin changes voltage. Four hardware blocks stand between that and your handler running, and you must configure all four. Miss one and nothing happens.

GPIO → SYSCFG → EXTI → NVIC → CPU core

Let's walk it left to right, writing the code as we go.

6.1 Stage 1: GPIO, get the signal into the chip

The GPIO ports can offer up to 96 port-bit signals to the next stage. For a pin's signal to be forwarded at all, its **input buffer must be enabled**, which happens when the pin is configured as digital input, digital output, or alternate function. It is *disabled* for analog and oscillator functions.

Listing 2: Step 1 of 4. Book Listing 4.3.

```
RCC->AHBENR |= RCC_AHBENR_GPIOCEN;                    /* clock the port */
MODIFY_FIELD(GPIOC->MODER, GPIO_MODER_MODER7,  ESF_GPIO_MODER_INPUT);
MODIFY_FIELD(GPIOC->MODER, GPIO_MODER_MODER13, ESF_GPIO_MODER_INPUT);
MODIFY_FIELD(GPIOC->PUPDR, GPIO_PUPDR_PUPDR7,  1);    /* pull-up */
MODIFY_FIELD(GPIOC->PUPDR, GPIO_PUPDR_PUPDR13, 1);
```

Recall Week 03: clock first, then mode, then the pull resistor. Recall Week 02: without the pull-up the pin floats and will generate interrupts from thin air.

Beware the trap

Note the corollary of "input buffer disabled for analog mode": **a pin configured as analog cannot generate an EXTI interrupt.** If you set `MODER` to `11` for an ADC input and then wonder why its interrupt never fires, that is why. It is not a bug, it is the mux.

6.2 Stage 2: SYSCFG, choose which port owns each line

Here is the design constraint that surprises everyone. The interrupt hardware supports only **sixteen** external interrupt lines, EXTI0 through EXTI15, one per *bit position*, shared across all ports.

So EXTI0 can be connected to PA0, or PB0, or PC0, or PD0, or PE0, or PF0. Exactly one of them. A six-input multiplexer.

The consequence, stated plainly

You cannot have PA0 and PB0 both generating interrupts at the same time. They compete for EXTI0.

You *can* have PA0 and PB1 both generating interrupts, because those use EXTI0 and EXTI1 respectively.

So when assigning interrupt-capable pins in a design, what matters is that no two of them share a bit number. This is a real constraint on schematic design, not a software detail.

The multiplexers are controlled by four registers, and CMSIS exposes them as a zero-indexed array, which is a classic off-by-one trap:

Register	Lines	CMSIS name	[15:12]	[11:8]	[7:4] / [3:0]
SYSCFG_EXTICR1	0–3	EXTICR[0]	EXTI3	EXTI2	EXTI1 / EXTI0
SYSCFG_EXTICR2	4–7	EXTICR[1]	EXTI7	EXTI6	EXTI5 / EXTI4
SYSCFG_EXTICR3	8–11	EXTICR[2]	EXTI11	EXTI10	EXTI9 / EXTI8
SYSCFG_EXTICR4	12–15	EXTICR[3]	EXTI15	EXTI14	EXTI13 / EXTI12

Field values select the port: 0 for port A, 1 for B, **2 for C**, 3 for D, and so on.

Listing 3: for line 13.]Step 2 of 4. Book Listing 4.4. Note EXTICR[3] for line 13.

```
RCC->APB2ENR |= RCC_APB2ENR_SYSCFGCOMPEN;   /* SYSCFG needs a clock too! */

/* SW1 is PC13 -> line 13 -> EXTICR4 -> CMSIS EXTICR[3], value 2 for port C
   */
MODIFY_FIELD(SYSCFG->EXTICR[3], SYSCFG_EXTICR4_EXTI13, 2);
/* SW2 is PC7  -> line 7  -> EXTICR2 -> CMSIS EXTICR[1] */
MODIFY_FIELD(SYSCFG->EXTICR[1], SYSCFG_EXTICR2_EXTI7,  2);
```

Beware the trap

Two traps in four lines.

First, **SYSCFG is on APB2, not AHB**, and it needs its own clock enable. Forget it and the mux writes vanish exactly as GPIO writes do without their clock. This is the second-most-forgotten clock enable in the course.

Second, the register is documented as `EXTICR1` through `EXTICR4` but accessed as `EXTICR[0]` through `EXTICR[3]`. Write `EXTICR[4]` for line 13 and you are indexing past the end of the array into whatever register follows. The book warns about this explicitly and it is worth the warning.

6.3 Stage 3: EXTI, decide what counts as an event

The signal now reaches the **Extended Interrupts and Events Controller**. Its job is to decide whether a change qualifies as an interrupt request. Four registers, each with one bit per line, so bit x always means line x:

Register	Function
EXTI_RTSR	Rising trigger select. 1 ⇒ a 0-to-1 edge can trigger
EXTI_FTSR	Falling trigger select. 1 ⇒ a 1-to-0 edge can trigger
EXTI_IMR	Interrupt mask. 0 ⇒ line disabled (masked). 1 ⇒ enabled
EXTI_PR	Pending. Set by hardware when a request occurs. **Cleared by writing 1**
EXTI_SWIER	Software trigger, for testing or software-generated events

Setting both `RTSR` and `FTSR` for a line triggers on *either* edge, which is what you want for a switch whose press and release both matter.

Listing 4: Step 3 of 4. Book Listing 4.5. Both edges, both switches.

```
EXTI->IMR  |= MASK(SW1_POS) | MASK(SW2_POS);   /* unmask lines 13 and 7 */
EXTI->RTSR |= MASK(SW1_POS) | MASK(SW2_POS);   /* trigger on rising      */
EXTI->FTSR |= MASK(SW1_POS) | MASK(SW2_POS);   /* and on falling         */
```

Clearing a pending flag by writing 1

`EXTI_PR` is **write-1-to-clear**. Writing a 1 to a bit clears it to 0. Writing a 0 does nothing.

This looks backwards and it is deliberate. It means the clear is a plain write of a single-bit mask, with no read first, so it is **atomic** for exactly the reasons `BSRR` was atomic in Week 03. An ISR clearing its own flag with a read-modify-write could race against a second interrupt arriving on a different line in the same register.

Beware the trap

If the handler does not clear `EXTI_PR`, the interrupt fires again immediately, forever. The pending bit is what requests the interrupt, so leaving it set means the NVIC re-requests the moment you return. Your program appears to hang, but it is actually executing your handler millions of times per second.

And it must be cleared with `EXTI->PR = MASK(pos);` using plain assignment, not `|=`. Both work, but `|=` reads first and is not atomic.

If you use HAL, `HAL_GPIO_EXTI_IRQHandler` clears the flag for you before calling your callback, which is why the manual's Lab 03 pattern works and why students who write a bare `EXTI0_IRQHandler` without HAL often hang on their first try.

6.4 Stage 4: NVIC, pick a winner and poke the CPU

NVIC

The **Nested Vectored Interrupt Controller**. It monitors every interrupt request line, and if several enabled interrupts are pending, it selects the one with the **highest priority** and directs the CPU to run its handler. "Nested" because a higher-priority interrupt can interrupt a running lower-priority handler. "Vectored" because it supplies the handler address from the vector table directly, with no software dispatch.

Enabling is done through `ISER` (interrupt set-enable) and `ICER` (interrupt clear-enable), each 32 bits with one bit per source. Write a 1 to bit N of `ISER` to enable IRQ N; write a 1 to bit N of `ICER` to disable it. Same write-1-to-act pattern, same atomicity reason.

You will almost never touch those directly, because CMSIS provides:

```
void      NVIC_EnableIRQ(IRQn);           /* enable  */
void      NVIC_DisableIRQ(IRQn);          /* disable */
void      NVIC_SetPriority(IRQn, prio);   /* set priority */
uint32_t  NVIC_GetPriority(IRQn);
void      NVIC_ClearPendingIRQ(IRQn);     /* clear a stale pending request */
```

Listing 5: Step 4 of 4. Book Listing 4.6. Note the order.

```
NVIC_SetPriority(EXTI4_15_IRQn, 3);      /* yours: EXTI0_IRQn etc. */
NVIC_ClearPendingIRQ(EXTI4_15_IRQn);     /* discard anything stale  */
NVIC_EnableIRQ(EXTI4_15_IRQn);           /* now let it through      */
```

Beware the trap

The order in those three lines matters. Configuring EXTI in step 3 can itself set a pending bit, because merely enabling edge detection on a pin that is already at some level can register as an event. If you enable the NVIC first, you take a spurious interrupt before your program is ready.

So: **set priority, clear pending, then enable.** Always in that order, and always last in the initialisation sequence.

7 Deep dive 6: Priority

Every exception source has a priority that decides which of several simultaneous requests is handled first.

The counterintuitive part

> **Lower priority number means higher urgency.** Priority 0 beats priority 3. Think of it as a ranking, first place and third place, not as a score.

Some exceptions have priorities fixed in hardware: reset, NMI, and HardFault always win, in that order, which is why a HardFault can always report a problem no matter what else is happening. Everything else is configurable.

Book board vs your board

> This is the second place your board genuinely differs.
>
> **Book (Cortex-M0, ARMv6-M):** 2 priority bits, giving **4 levels**. The priority registers `IPR0`–`IPR7` hold an 8-bit field per source, but only the top 2 bits are implemented, so the usable values are 0, 64, 128, and 192. CMSIS hides this: you pass 0, 1, 2, or 3 to `NVIC_SetPriority` and it shifts left by 6 for you.
>
> **Yours (Cortex-M4 on STM32F3): 4 priority bits**, giving **16 levels**, 0 through 15. CMSIS shifts left by 4. So the book's "priority 3 is the lowest urgency" becomes, on your part, "priority 3 is fairly urgent, and 15 is the lowest".
>
> **Copying the book's priority numbers onto your board therefore changes their meaning.** A design where the book intended "least urgent" becomes moderately urgent on yours, and the interrupt you wanted to win may lose.
>
> The M4 also splits priority into *group* (preemption) and *sub-priority* fields via `NVIC_SetPriorityGrouping`, so only group priority determines whether one interrupt can preempt another. HAL's `HAL_NVIC_SetPriority(IRQn, group, sub)` takes both, which is why its signature has an argument the book's version does not.

In the lab

> Priority is not decoration in your self-balancing robot, it is a design decision with a right answer.
> The control loop's timer interrupt must fire on schedule or the PID derivative term is computed from a wrong time step. A UART receive interrupt used for telemetry can afford to be late by a millisecond. So the timer gets a numerically lower priority value than the UART.
> Get that backwards and a burst of telemetry characters delays your control loop, the sample interval jitters, and the robot develops an oscillation you will spend an evening blaming on your PID gains.

8 Deep dive 7: PRIMASK, and masking interrupts correctly

Last stage of the chain: the CPU core itself can be told to ignore interrupts entirely.

The `PRIMASK` register's `PM` flag, bit 0, does this. `PM` $= 0$ means interrupts of configurable priority are recognised; `PM` $= 1$ means they are ignored. Reset clears it to 0, so interrupts are enabled from the start. The underlying instructions are `CPSID` (disable) and `CPSIE` (enable).

```
1 void      __enable_irq(void);                 /* clear PM   */
2 void      __disable_irq(void);                /* set PM     */
```

```
uint32_t __get_PRIMASK(void);
void     __set_PRIMASK(uint32_t x);
```

Note that `PRIMASK` does not stop NMI or HardFault. That is what "non-maskable" means.

8.1 The bug everybody writes once

You need a short sequence to run without interruption. The obvious code:

Listing 6: Wrong. Looks fine, is not.

```
__disable_irq();
/* critical section */
__enable_irq();           /* BUG: unconditionally enables */
```

Ask what happens if interrupts were *already* disabled when this function was called, because it was itself called from inside another critical section. Your `__enable_irq()` at the end switches them back on, and the caller's critical section silently loses its protection from that point on. The caller has no idea. The bug appears far away from its cause.

Listing 7: Correct. Book Listing 4.7. Save, disable, restore.

```
void my_function(void) {
    uint32_t masking_state;

    /* non-critical work ... */

    masking_state = __get_PRIMASK();    /* remember how things were */
    __disable_irq();

    /* critical section ... */

    __set_PRIMASK(masking_state);       /* put them back as they were */

    /* more non-critical work ... */
}
```

The general principle

A function that changes global machine state must **restore the previous state**, not impose a default. This applies to interrupt masking, to peripheral clock enables, and to anything else shared. "Save, modify, restore" composes correctly under nesting; "modify, then set to a known value" does not.

Beware the trap

`__disable_irq()` is a blunt instrument: it stops *every* maskable interrupt, including your control loop's timer. Keep critical sections as short as physically possible, because their length adds directly to the worst-case latency of every interrupt in the system.

Recall the responsiveness definition from Week 01: worst case, not average. A single long critical section is enough to blow a deadline that the rest of your system meets comfortably. On your F303 there is a finer tool, `BASEPRI`, which masks only interrupts below a given priority and leaves urgent ones running. The Cortex-M0 does not have it, which is why the book does not mention it.

9 Tying it back

We asked what happens between a switch closing and a function you never called starting to run. The answer is a chain, and now you can name every link.

The pin's input buffer forwards its level to SYSCFG, which is a set of six-to-one multiplexers deciding, for each of sixteen bit positions, which port owns that interrupt line, which is why PA0 and PB0 cannot both be interrupt sources. EXTI then decides what counts as an event, watching for rising edges, falling edges, or both, and records a request by setting a pending bit that your handler must clear by writing a 1, or the interrupt repeats forever. The NVIC picks the highest-priority pending request, where lower number means more urgent, and directs the CPU. The CPU core, unless PRIMASK says otherwise, finishes its instruction, pushes eight registers, switches to Handler mode on MSP, loads the handler address from the vector table, and starts executing, all in sixteen cycles on the book's M0 and twelve on your M4. Because the eight registers pushed are exactly the caller-saved set, your handler can be an ordinary C function with no special annotation. On the way out, `BX LR` with the magic `EXC_RETURN` value in `LR` tells the hardware to unwind, and no dedicated return-from-interrupt instruction was ever needed.

Four configuration steps, in order: GPIO, SYSCFG, EXTI, NVIC. Miss any one and nothing happens, silently.

Which leaves a problem we have carefully avoided. That handler can run *between any two instructions* of your main program, and both of them touch the same global variables. Recall the `g_flash_LED` from Week 05, written by the ISR and read by the tasks. What if the ISR fires halfway through the task's read? That is Week 08, and it is the week where `volatile` stops being enough.

Cheat sheet: Week 07

Vocabulary

An **interrupt is a type of exception**. **IRQ** = the hardware request signal. **ISR / handler** = the function that runs.
Asynchronous: can occur between almost any two instructions.
Thread mode = normal, uses MSP or PSP. **Handler mode** = servicing an exception, **always MSP**.
`SPSEL` in `CONTROL` picks MSP (0, the reset default) or PSP (1) in Thread mode. Bare-metal uses MSP only.

Entry sequence, 16 cycles on M0 / 12 on M4

1. Finish current instruction. `LDM`, `STM`, `PUSH`, `POP`, `MULS` are **abandoned and restarted**.
2. Push 8 registers: `xPSR`, `PC`, `LR`, `R12`, `R3`, `R2`, `R1`, `R0`. *Exactly the caller-saved set, which is why an ISR is a normal C function.*
3. Switch to Handler mode, use MSP. 4. Load PC from vector table. 5. Load `LR` with `EXC_RETURN`. 6. Load `IPSR` with exception number.

Exit: `BX LR` with the `EXC_RETURN` code. **ARMv6-M has no return-from-interrupt instruction.**
`0xFFFFFFF1` Handler/MSP · `0xFFFFFFF9` Thread/MSP · `0xFFFFFFFD` Thread/PSP

Vector table

Array of handler addresses at the bottom of memory. `0x00` holds the **initial SP**, not a handler. `0x04` Reset, `0x08` NMI, `0x0C` HardFault, `0x3C` SysTick, `0x40` = IRQ0.
vector # = 16+ **IRQ #**. CMSIS names use IRQ numbers.
Defined with `DCD` in `startup_stm32f303xc.s`. **Misspell a handler name and it is silently never called**, because a weak default handler loops forever. Symptom: board hangs when the interrupt fires.

The four-stage chain, and the four code blocks

GPIO → SYSCFG → EXTI → NVIC → CPU

1. **GPIO**: clock the port, `MODER` to input, `PUPDR` for the pull. Input buffer is off in **analog** mode, so an analog pin **cannot** generate EXTI.
2. **SYSCFG**: **needs its own APB2 clock**. `EXTICR[0..3]` are documented as `EXTICR1..4`, so **line 13 is** `EXTICR[3]`. Field value = port index (C = 2).
3. **EXTI**: `IMR` unmask, `RTSR` rising, `FTSR` falling, both for either edge. `PR` pending, **write 1 to clear**.
4. **NVIC**: `SetPriority`, then `ClearPendingIRQ`, then `EnableIRQ`. **In that order**, or you take a spurious interrupt.

Sixteen lines, shared across all ports

One EXTI line per **bit position**, not per pin. EXTI0 serves PA0 *or* PB0 *or* PC0 ... pick one. **PA0 and PB0 cannot both interrupt. PA0 and PB1 can.** A schematic-design constraint.

Handler grouping differs by part. F091: `EXTI0_1`, `EXTI2_3`, `EXTI4_15`. **F303**: `EXTI0`, `EXTI1`, `EXTI2_TSC`, `EXTI3`, `EXTI4`, `EXTI9_5`, `EXTI15_10`. *The book's `EXTI4_15_IRQHandler` does not exist on your board.*

Priority

Lower number = higher urgency. Priority 0 beats priority 3. Fixed for reset, NMI, HardFault.

Book (M0): **2 bits, 4 levels**, raw values 0/64/128/192, CMSIS shifts by 6, pass 0–3.

Yours (M4/F3): **4 bits, 16 levels**, CMSIS shifts by 4, pass 0–15.

So the book's priority numbers change meaning on your board. Its "3 = least urgent" is mid-range on yours.

M4 also splits group (preemption) from sub-priority, hence HAL's three-argument `HAL_NVIC_SetPriority`.

Masking, correctly

`PRIMASK.PM` = 1 ignores all maskable interrupts. Reset value 0. Does not affect NMI or HardFault.

Wrong: `__disable_irq(); ...; __enable_irq();` — silently re-enables inside a caller's critical section.

Right: `s = __get_PRIMASK(); __disable_irq(); ...; __set_PRIMASK(s);`

Principle: **save, modify, restore**. Nests correctly. Setting a default does not.

Critical-section length adds directly to **worst-case latency of every interrupt**. Keep them tiny. Your M4 has `BASEPRI` for selective masking; the M0 does not.

Likely exam prompts from this chapter

· List the four hardware stages between a pin change and an ISR, and what you configure in each. *Near certain.*

· Why can PA0 and PB0 not both be interrupt sources?

· Which registers are stacked on exception entry, and why exactly those?

· What happens if a handler fails to clear `EXTI_PR`?

· Write the correct code for a critical section and explain why the naive version is wrong.

· Given two priority numbers, state which interrupt is serviced first.